Networks & Distributed Programming

The Transport Layer: Reliable Data Transfer

Lecture 15



UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II



DIETI

UNINA

Transport Layer
Reliable Data Transfer Problem

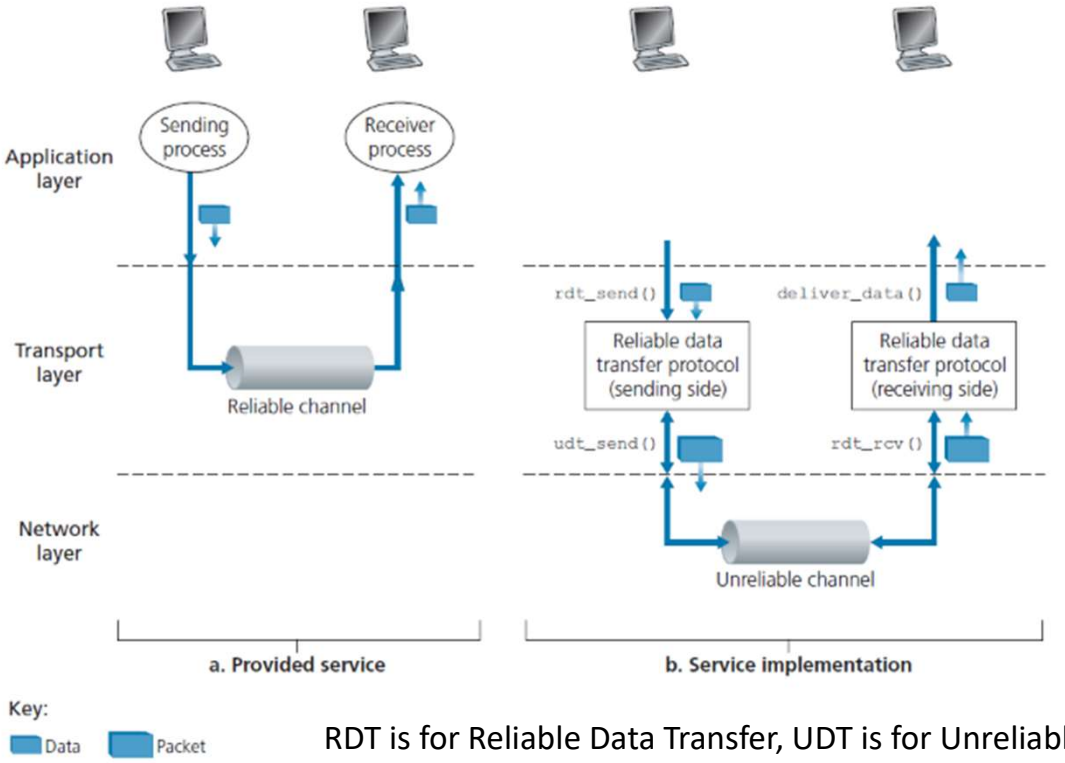
Reliable Data Transfer: The General Problem

- **Goal:** ensure that data is delivered correctly, completely, and in the original order.
- **Challenge:** the underlying communication channel may **corrupt bits or lose packets**.
- **Abstraction:** provide upper layers with a **reliable communication service**, even when the underlying channel is unreliable.
- **Scope:** a fundamental problem at the **link, transport, and application layers**, not specific to TCP.

Key question: How can a reliable data transfer protocol be built on top of an unreliable channel?

Transport Layer
Reliable Data Transfer Problem

Reliable Data Transfer: The General Problem



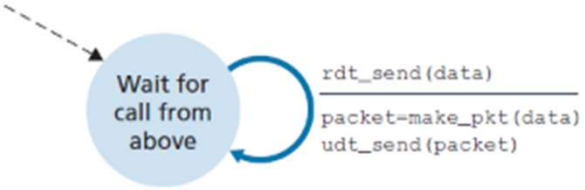
RDT is for Reliable Data Transfer, UDT is for Unreliable Data Transfer

Transport Layer
Reliable Data Transfer Problem

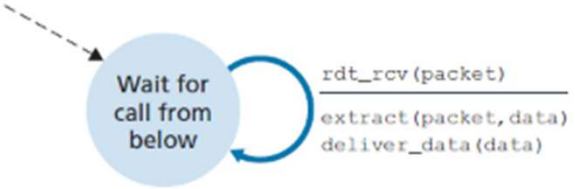
Reliable Data Transfer: The General Problem

Let's first consider an ideal scenario where the underlying channel is perfectly reliable.

- the sender simply transmits packets
- the receiver delivers them to the upper laye
- No additional mechanisms are needed



a. rdt1.0: sending side



b. rdt1.0: receiving side

Transport Layer

An announcement on an unreliable channel

- Let's assume we are at the railway station waiting for the train number 6 which should arrive on platform 5.
- The train changes platform, it will arrive on platform 9 instead of platform 5, a message is sent from station control to communicate it:

  "Train 6 will arrive on platform 9"

- **What happens if the communication is unreliable:**
  - Words could be **lost**: "Train %&! will arrive on platform 9"
  - Words could be **altered**: "Train 7 will arrive on platform 9"
  - Words could be **swapped**: "Train 9 will arrive on platform 6"
  - Words could be **duplicated**: "Train 66 will arrive on platform 9"

- It is possible you can understand that the message is wrong (for example in case 1), but you may also take the wrong train or miss the right one.



Transport Layer
Reliable Data Transfer

- Error control (e.g., through checksum) allows receiver at least to understand **if the message is corrupted**, but this is not enough. Reliable data transfer **is still one of the main problem of networking**.
  - The problem is not only addressed at the transport layer, but also at the link layer and (often) at the application layer.

- In a reliable channel 3 elements must be guaranteed:
  1. None of the transmitted bits are corrupted (flipped from 0 to 1, or vice versa).
  2. None of the transmitted bits are lost or repeated.
  3. All bits are delivered in the exact order in which they were sent.

- In general, we must assume the lower-level **network layer to be unreliable**.
  - This is a realistic assumption, for example, TCP is a reliable data transfer protocol that **is implemented on top of an unreliable (IP) end-to-end network layer**.

Transport Layer

Reliable Data Transfer Problem

Unidirectional Data Transfer

- We consider **data transfer in one direction only**, from sender to receiver.
- However, **control packets (ACKs/NACKs)** must also travel in the opposite direction.
- Both sender and receiver use unreliable data send to transmit packets over the unreliable channel.
- **Unidirectional data flow ≠ Unidirectional communication.**

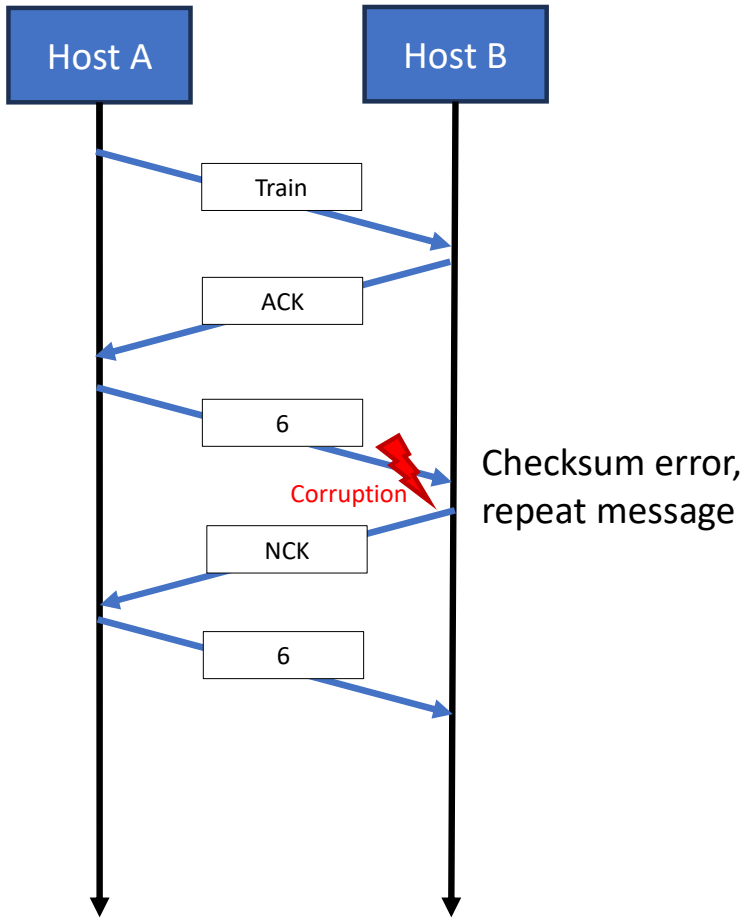
Transport Layer

Reliable Data Transfer: Packet Corruption and Stop-and-wait

- Let's assume we have **a channel where bits may only be corrupted** (not lost).
  - Corruption of bits is a typical problem, which is **mainly due to the physical components** of a network as a packet is transmitted, propagated, or buffered several time during the communication.

- **Stop-and-wait**: each message to be acknowledged before sending a new one:
  - **Positive acknowledgments** (ACK) means that the message has been received intact.
  - **Negative acknowledgments** (NACK) means that an error occurred, so the message must be repeated.

- In a computer network, **protocols based on retransmission are also known as ARQ** (Automatic Repeat reQuest) protocols.



Transport Layer

Reliable Data Transfer: Automatic Repeat reQuest

What an ARQ protocol needs

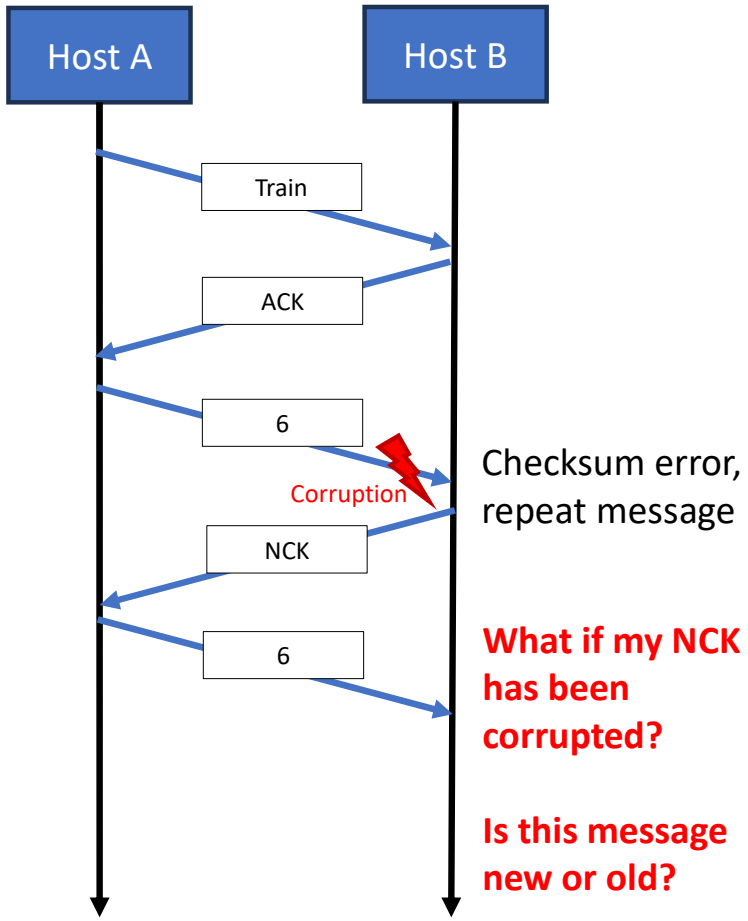
- **Error detection:** extra bits that let the receiver notice the message changed — the checksum you have already met.
- **Receiver feedback:** the sender is away, the only way to know is to be told.
- **Retransmission:** whatever arrived in error is sent again. There is no other repair.

In principle the feedback is one bit: a value for *yes* and one for *no*. None of the three is optional — and, as it turns out, the three together are still not enough.

Transport Layer

Reliable Data Transfer: Stop-and-wait

- There are a couple of **issues** with this approach:
  - What happens if the **ACK/NACK message is corrupted** itself?
  - How can Host B be sure that **a message is the repetition instead of a new message**?

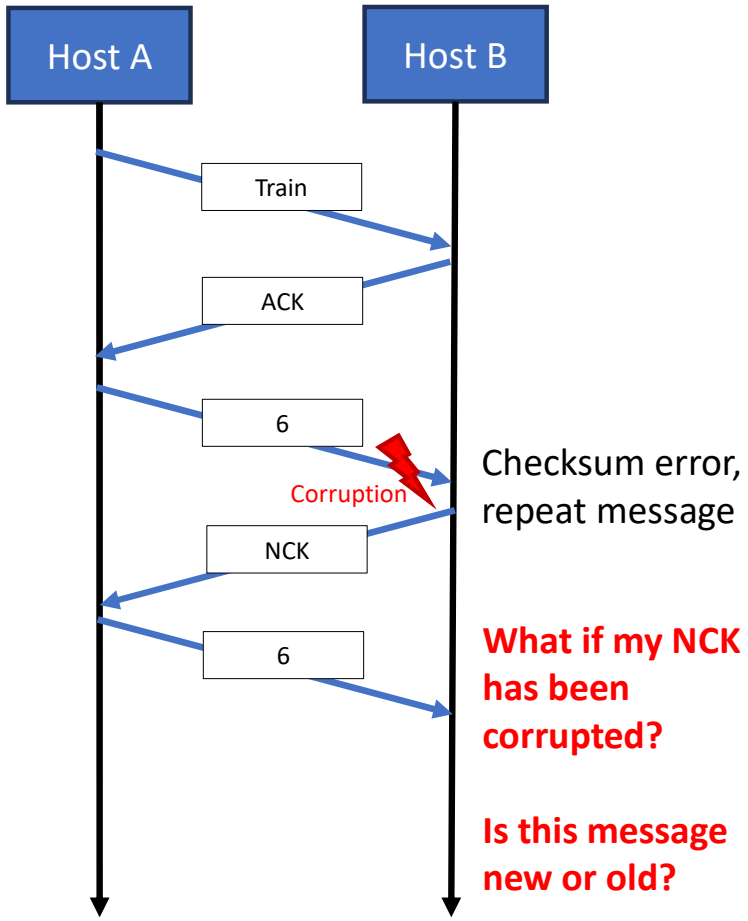


Transport Layer

Reliable Data Transfer: Stop-and-wait

- There are a couple of **issues** with this approach:
  - What happens if the **ACK/NACK message is corrupted** itself?
  - How can Host B be sure that **a message is the repetition instead of a new message**?

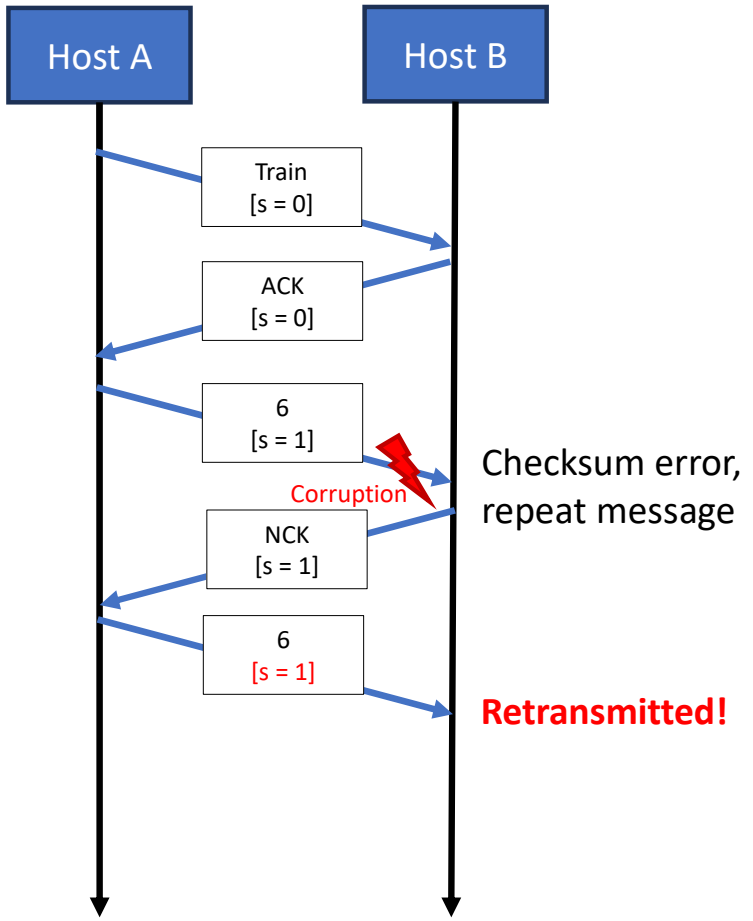
- **Sequence number**: add a new field to the headers of packets which specifies the order in which packages should be received.



Transport Layer

Reliable Data Transfer: Stop-and-wait

- In a stop-and wait approach we always have **one message** on the wire.
- There is no need to specify the whole sequence of packets, we just need to **differentiate between the current packet and the previous one**.
- Therefore, in this case, **we just need to add one bit** (*s = 0/1*) to the headers of packets.

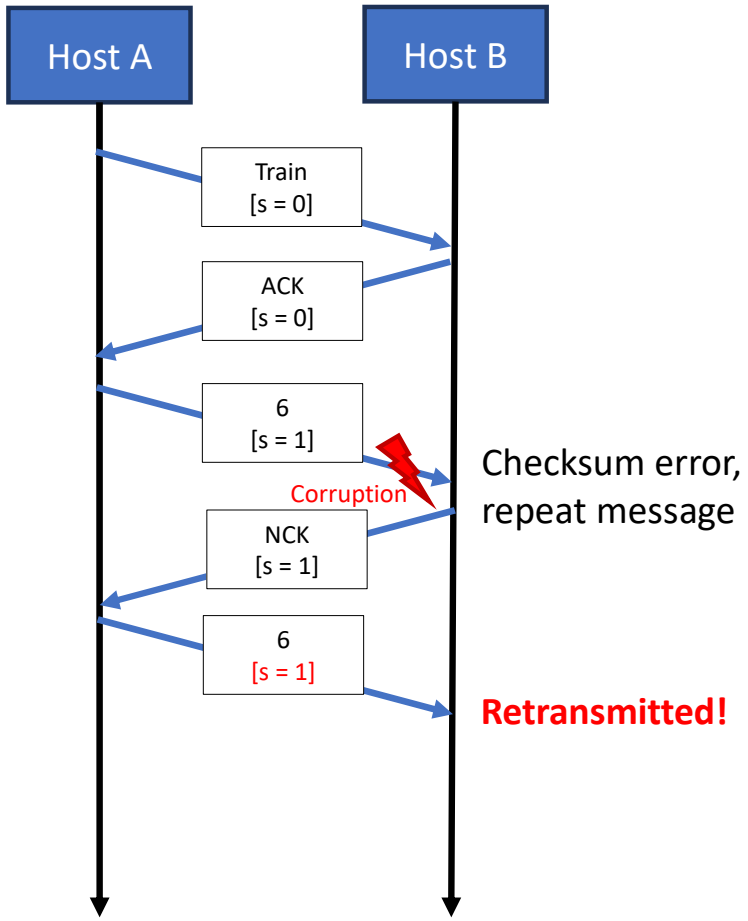


Transport Layer

Reliable Data Transfer: Stop-and-wait

- In a stop-and wait approach we always have **one message** on the wire.

- Add a field s = 0/1 to the header: it alternates
- A repeated value means a repeated message.
- The acknowledgement carries the same bit, so an answer says which message it answers.



Transport Layer

Reliable Data Transfer: Stop-and-wait

Numbered ACKs make NACKs Unnecessary

- A **NACK** requests a retransmission.
- A **numbered ACK** identifies the last successfully received packet.
- If the sender receives an ACK for the previous packet, the current packet has not been acknowledged.
- The receiver can respond to both **corrupted and duplicate packets** by repeating the last ACK.

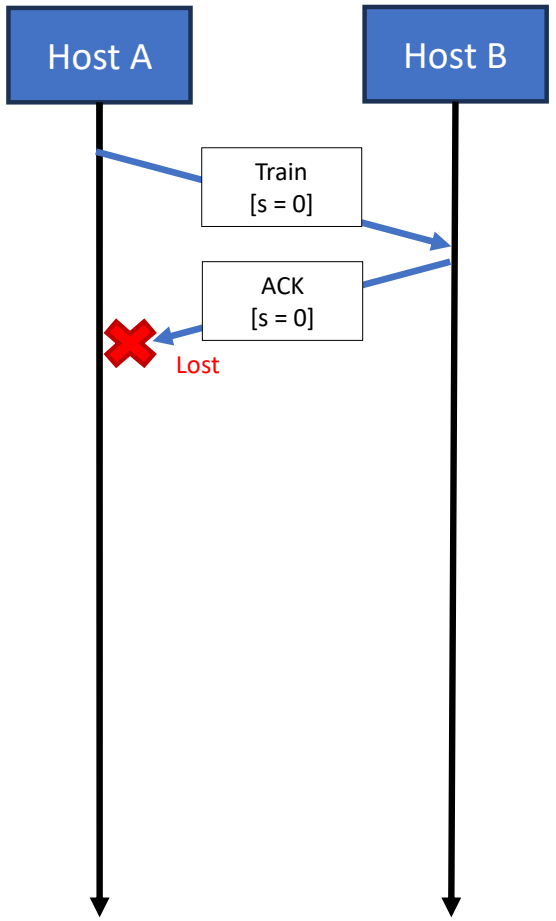
Key idea: ACKs can provide both positive and negative feedback, eliminating the need for explicit NACKs.

Transport Layer

Reliable Data Transfer: Packet Loss and Stop-and-wait

- Let's assume we now have **a channel where packets may also be lost**.
  - This is also quite reasonable as **network devices may have buffer overflow** in case of intense traffic.

- Here we have a clear problem with stop-and-wait approach: the loop. **If one message is lost, hosts will never send a new one**.
  - Notice that this happens whether we lose message or acknowledgment, as in both cases host A will not be triggered to send a new message.



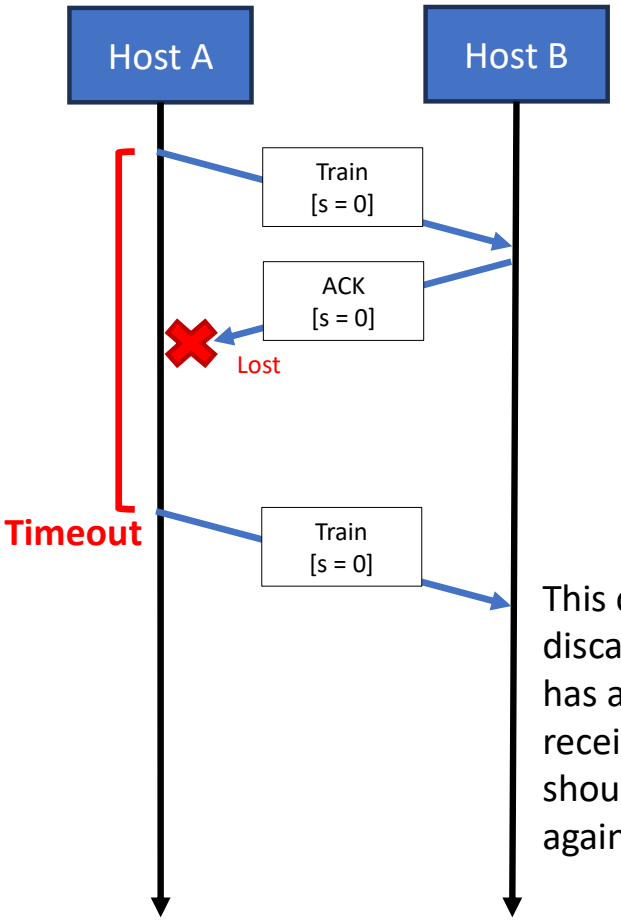
Transport Layer

Reliable Data Transfer: Packet Loss and Stop-and-wait

- **Timeout**: a very simple yet effective solution is to **add a timeout on the sender-side** that, once expired, allows the Host to try again.

- The duplicate is recognised by its sequence
  - number, discarded, and acknowledged again.
  - A DNS query over UDP does exactly this: no answer in time, ask once more.

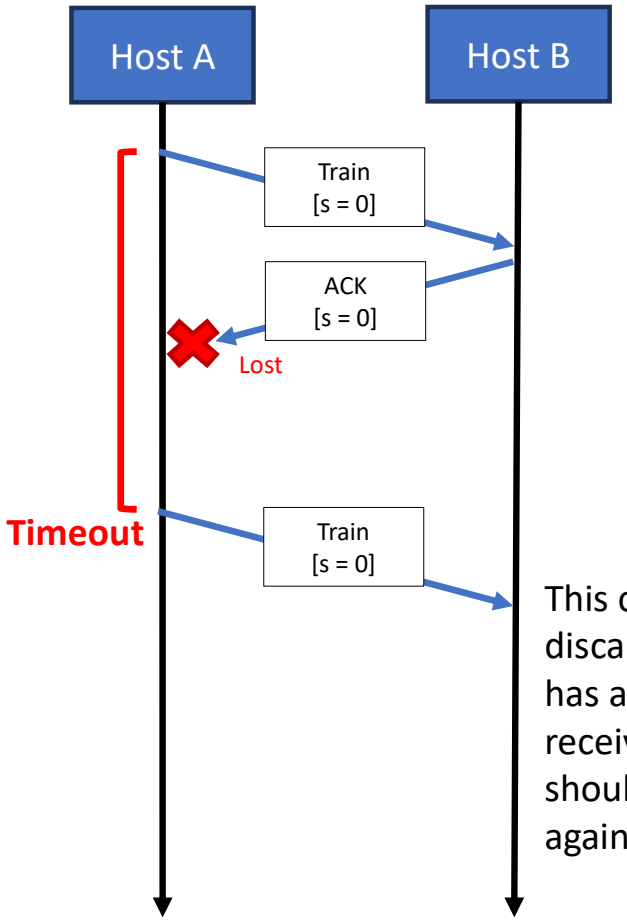
- This works **both in the case of packet loss and in acknowledgment loss**.
  - In case of duplicate the receive has just to discard the packet.



Transport Layer

Reliable Data Transfer: Packet Loss and Stop-and-wait

- The challenge here **is how to estimate a suitable timeout** (it depends on the RTT).
  - Long timeout causes communication to be slowed.
  - Too short timeout causes the packets to overlap.
- We can say that a reasonable timeout should be somehow longer than the RTT (Round-Trip Time).
- Timeout saves stop-and-wait from loop, but the **performance are quite bad**…



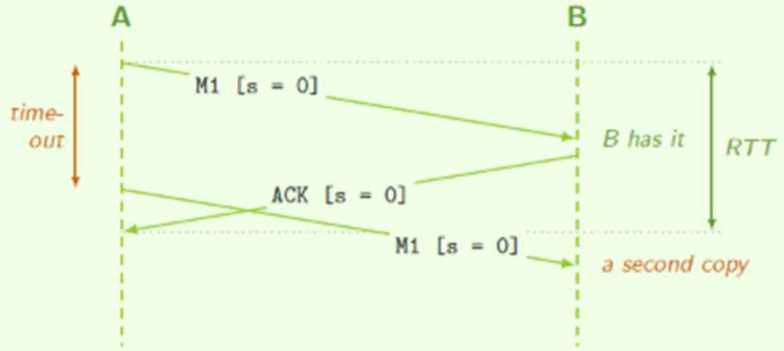
Transport Layer

Reliable Data Transfer: Packet Loss and Stop-and-wait

- **A Timeout** shorter than RTT

The timer is shorter than the round trip. What can the sender do right?

Nothing. The acknowledgement cannot arrive before the timer fires, so every message is retransmitted — on a channel that is losing nothing.



nothing was lost, nothing was corrupted — and B has two copies

This is the 50 ms timer on the 80 ms path: everything goes twice, and the sequence number at the receiver is the only reason the program still works.

Transport Layer

Reliable Data Transfer: Packet Loss and Stop-and-wait

From a perfect channel to a real one

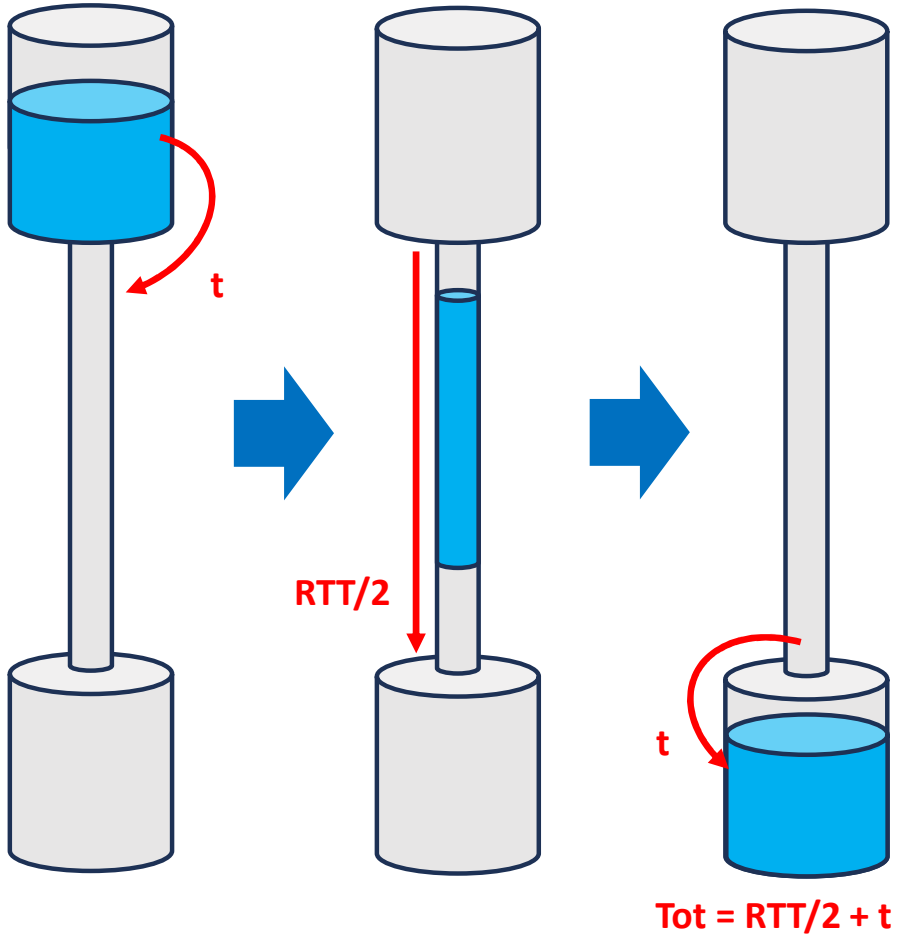
What the channel does	What the protocol must add
nothing goes wrong	nothing: make the packet and send it
bits can be corrupted	checksum, acknowledgements, retransmission
the acknowledgement too	a sequence number, so a copy is recognisable
packets can be lost	a timer at the sender, and the retransmission it triggers

The last row is the alternating-bit protocol: checksum, one-bit sequence number, numbered acknowledgements, one timer. It is correct on any channel we have described — and we are about to measure how slow.

Transport Layer

Reliable Data Transfer: Water Example

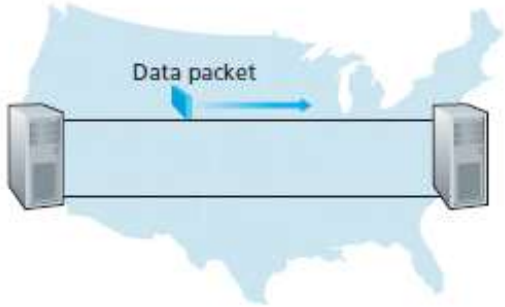
- We can see **data transfer as a water flow problem**:
  - There is **the time (t) to move the water from the tank to the tube**. While entering **the water is also moving** toward the tube.
  - After **t all the water from the tank is inside the tube** (the initial tank is empty as well as the destination tank).
  - It **takes a certain time (RTT/2) for the water to flow** into the tube.
  - Finally, **the water arrives at the destination tank** and (by assuming same download/upload time) it takes the same time t to pour out of the tube (RTT/2 + t to receive the last drop).



Transport Layer

Reliable Data Transfer: Performance of Stop-and-Wait

- Let's consider the idealized case of **two hosts located on the opposite coasts** of the United States.



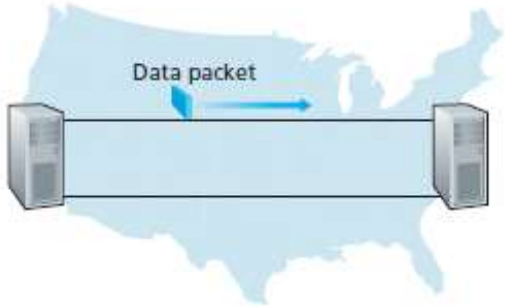
- The speed-of-light RTT between these two end systems **is approximately 30 milliseconds** (0.03 sec.). Let's assume to have:
  - A channel with a transmission rate (R) of 1 Gbps (10^9 bits per second).
  - A packet size (L) of 1000 bytes (8000 bits).
- The time (t) needed to transmit the packet to the channel is:

$$t = \frac{L}{R} = \frac{8000}{10^9}$$ (8 microseconds)

Transport Layer

Reliable Data Transfer: Performance of Stop-and-Wait

- Let's consider the idealized case of **two hosts located on the opposite coasts** of the United States.



- In a **stop-and-wait** protocol:
  - the **sender begins sending** the packet at t_0,
  - the **last bit enters the channel** at t_0 + 0.000008 sec.,
  - the **packet takes 0.015 sec. to reach** the receiver,
  - the **last bit is received** at t = RTT/2 + L/R = 0.015008 sec.,
  - assuming that ACK packets are extremely small (their **transmission time can be neglected**) the ACK emerges back at the sender at t = RTT + L/R = 0.030008 sec.
- In 0.030008 sec. of total transmission time, the sender was waiting almost all the time (99.973% of the time).

Transport Layer

Reliable Data Transfer: Performance of Stop-and-Wait

The formula

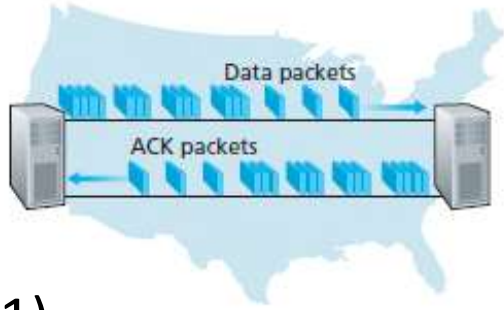
One message, one acknowledgement, then the next. On a path with a 100 ms round trip, a 1 KB message and a 100 Mbps link: $L/R = 80\ \mu s$, so $U = 0.0008$ and the chat moves about 80 kbps.

- On loopback and on the laboratory network the round trip is a fraction of a millisecond, so one never sees it.
- Ten messages a second is fine for a chat. It is not a way to move a file, and it is not what TCP does.

Transport Layer

Reliable Data Transfer: Performance of Pipelining

- **Pipelining**: instead of stop-and-wait, the sender is **allowed to send multiple packets** without waiting for acknowledgments.

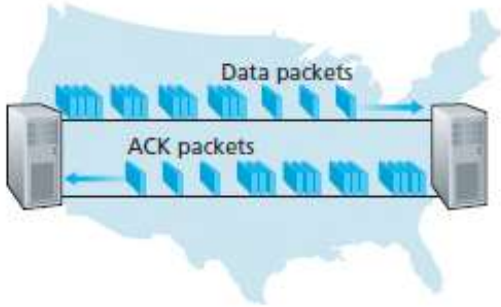


- In pipelining approaches (e.g., 3 packets instead of 1):
  - the **sender begins sending the 3 packets** at t_0,
  - the **last bit of the last packet enters the channel** at t_0 + 0.000024 sec.,
  - the **packets take 0.015 sec. to reach** the receiver,
  - the **last bit is received** at t = RTT/2 + L/R = 0.015024 sec.,
  - assuming that ACK packets are extremely small (their **transmission time can be neglected**) the ACK emerges back at the sender at t = RTT + L/R = 0.030024 sec.
- In 0.030024 sec. **the sender was waiting 0.035% less** (99.920% instead of 99.973%).

Transport Layer

Reliable Data Transfer: Performance of Pipelining

- **Pipelining**: instead of stop-and-wait, the sender is **allowed to send multiple packets** without waiting for acknowledgments.

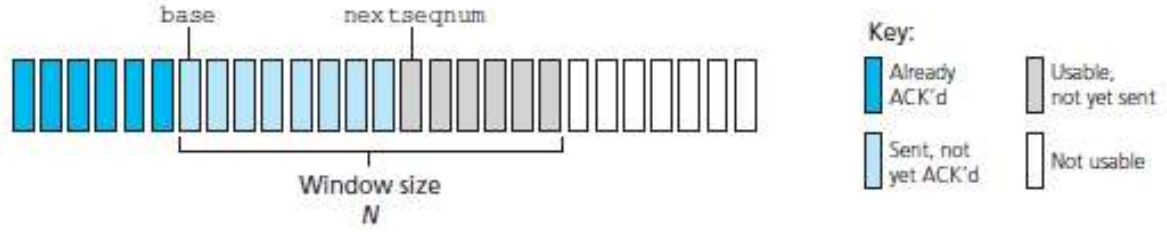


- In pipelining approaches:
  - The **range of sequence numbers must be increased** since each in-transit packet (not counting retransmissions) must have a unique sequence number and there may be multiple, in-transit, unacknowledged packets.
  - We **need buffers to store incoming packets** since we don't know if packets are correct or there are "holes" in the transmission.
- There are 2 basic protocols using pipelining:
  - **Go-Back-N**.
  - **Selective Repeat**.

Transport Layer

Reliable Data Transfer: Go-Back-N

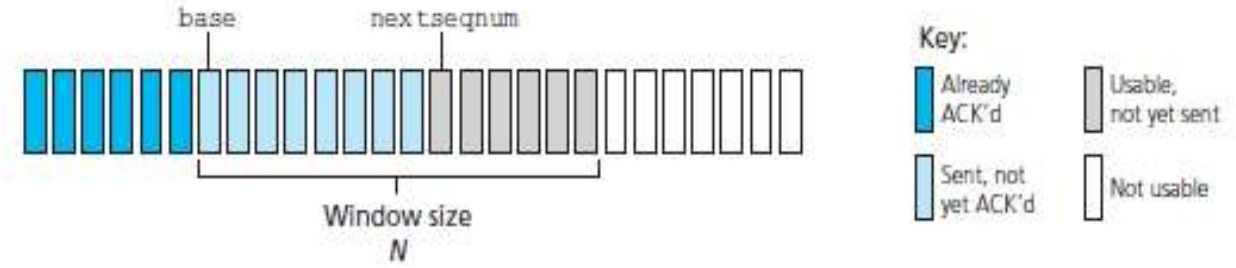
- **Go-Back-N (GBN)** protocol (aka sliding-window protocol): the sender is allowed to transmit multiple packets (when available) **without waiting for an acknowledgment** but is constrained to have **no more than N unacknowledged packets**.



- Base: is the **sequence number of the oldest unacknowledged packet**.
- Nextseqnum: is the **smallest unused sequence number** (next to be sent).
- We have that:
  - Packets in [0, base-1] have been **transmitted and acknowledged**.
  - Packets in [base, nextseqnum-1] have been **sent but not yet acknowledged**.
  - Packets in [nextseqnum, base+N-1] **can be sent**.
  - Packets in [base+N, +inf] **cannot be used** until a new acknowledgment is received.

Transport Layer

Reliable Data Transfer: Go-Back-N - sender



- **Cumulative ACKs:**
  - acknowledgement for packet *n* means that everything up to and including *n* arrived correctly
  - **A lost acknowledgement repairs itself** — ACK 5 makes ACK 3 and ACK 4 unnecessary, and the window jumps forward.
- **Single timer:**
  - it belongs to the oldest unacknowledged packet, not to each packet.
  - On **timeout the sender resends all the outstanding packets** (Go-Back-N).
- **Sliding window:**
  - When an **acknowledgement arrives** the window advances and **the timer is restarted**

Transport Layer

Reliable Data Transfer: Go-Back-N - receiver

The **receiver** keeps one number:

- Its entire state is expectedseqnum: the sequence number of the next packet it is willing to deliver
- **Correct and in order**: deliver it to the application, acknowledge it, move on by one.
- **Anything else**: discard it, and re-acknowledge the last in-order packet received.
- **No buffer:** A perfectly good packet that arrives early is thrown away exactly as if it had been damaged.

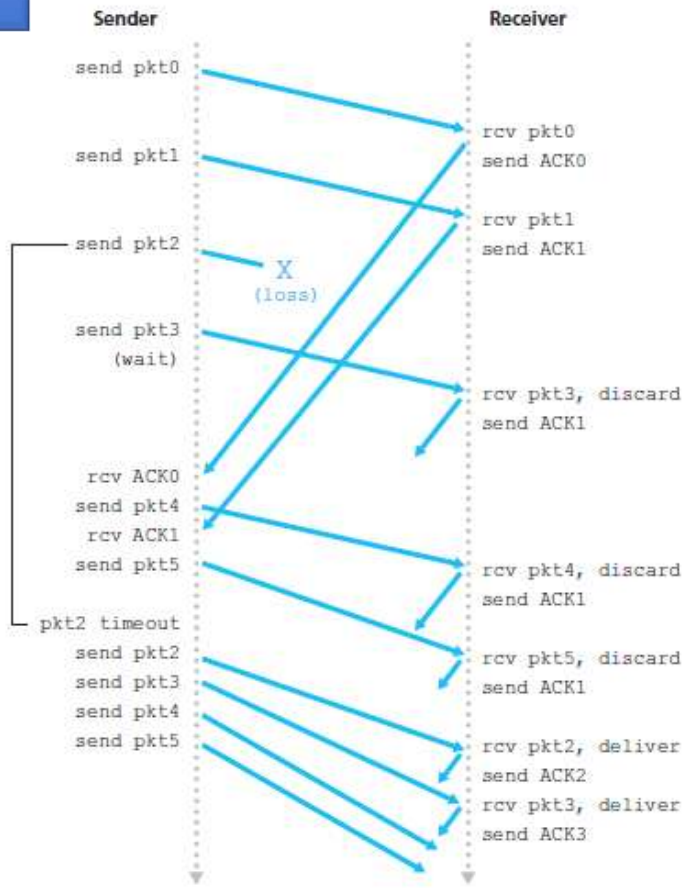
The complexity is deliberately all on one side: the sender keeps a window, the receiver keeps an integer.

Transport Layer

Reliable Data Transfer: Go-Back-N - receiver

- The receiver has **simply to discard out-of-order packets** (whether they are damaged or not) and to deliver to the upper level (app) in-order packets only.
  - Discarded packets (not acknowledged) will be eventually retransmitted by the sender.

- With this approach good packets are also discarded but the overall process is quite simple:
  - the **sender must maintain the indices of the window**,
  - the **receiver only needs to maintain the sequence number** of the next in-order packet.

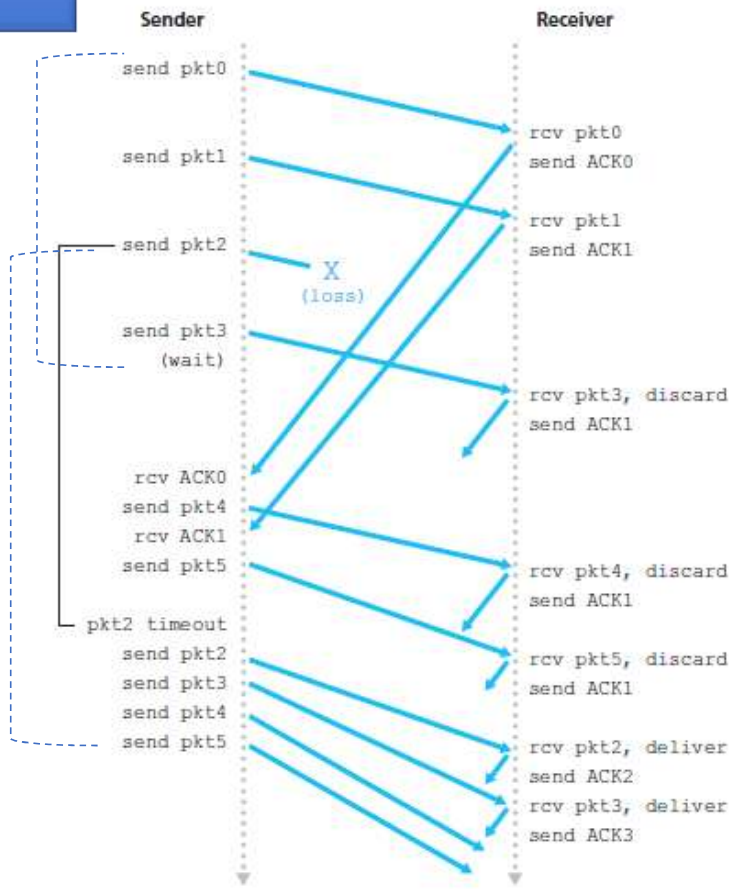


Since pkt2 is lost, all pkt3, pkt4, pkt5, must be discarded even if correct.

Transport Layer

Reliable Data Transfer: Go-Back-N – in action

windows-size = 4, (wait) on win full



Example:

- **Window of four**: packets 0 to 3 go out, then the sender has to wait for an acknowledgement to move the window.
- After send pk3 the windows is full (wait).
- **pkt2 is lost**: Packets 3, 4 and 5 arrive correct and are discarded, because 2 has not been delivered.
- **The receiver keeps answering ACK1**: the last thing it actually handed to the application.
- **The timer of pkt2 expires:** four packets go out again.

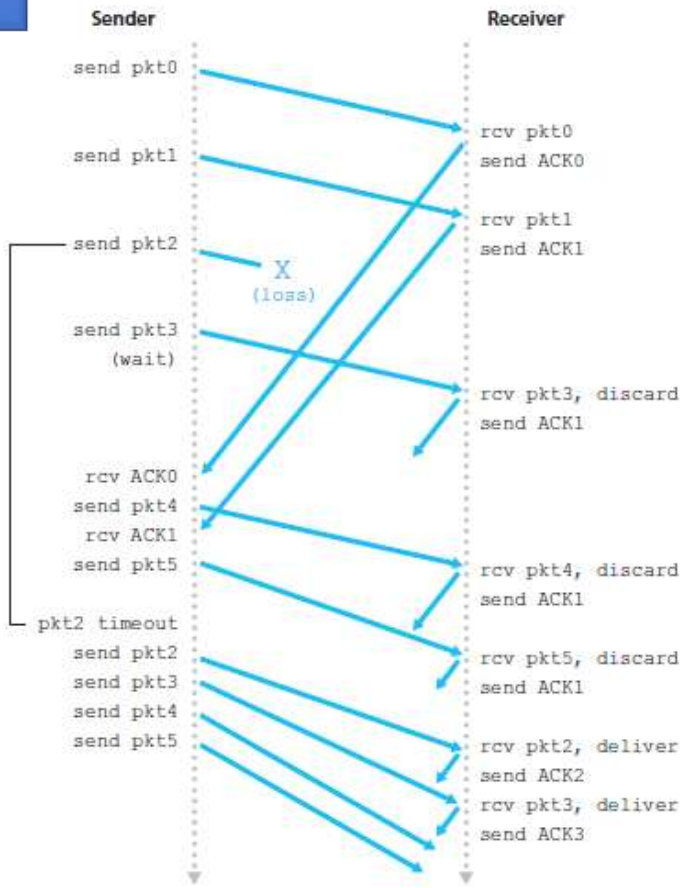
Since pkt2 is lost, all pkt3, pkt4, pkt5, must be discarded even if correct.

Transport Layer

Reliable Data Transfer: Go-Back-N — limits

- One loss, N retransmissions!
  - Three correct packets were thrown away. Who pays?
  - Sender, receiver and the channel

- **Chain-reaction**:
  - out-of-order packets are discarded forcing the sender to retransmit them.
  - the more the window holds, the more a single loss costs
  - Worst exactly where pipelining is worth most — a large window on a fast, long path.
  - The retransmission itself might be lost or damaged and thus even more retransmissions would be required.



Not only pkt2, but also pkt3, pkt4, pkt5, must be transmitted again.

Transport Layer

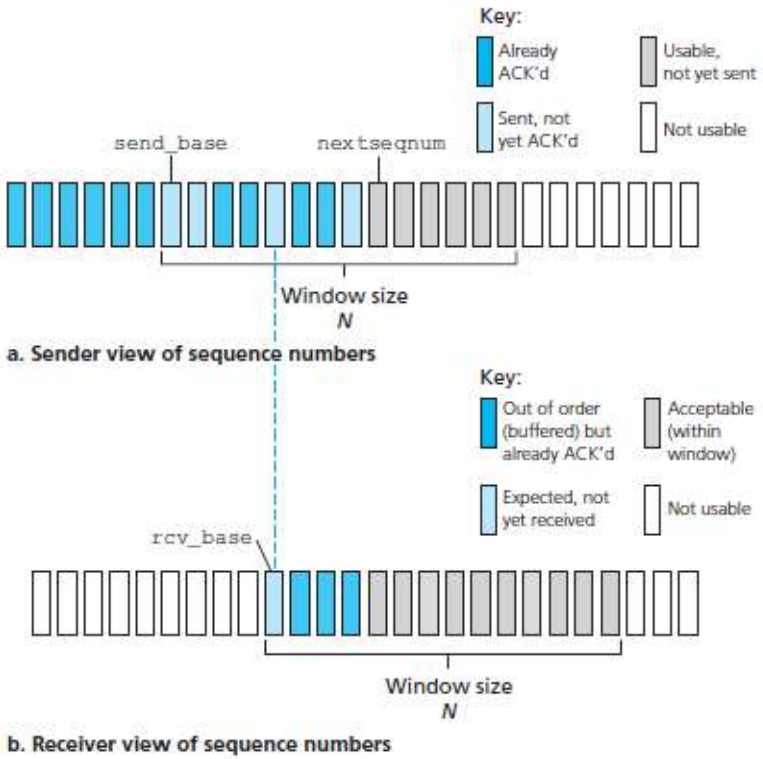
Reliable Data Transfer: Selective Repeat

Selective-repeat (SR) protocols:

- the sender retransmit only those packets that were likely received in error (lost or corrupted)
- Unlike GBN, out-of-order packets are buffered and acknowledged by the receiver instead of discarded.

Approach:

- Two windows of size N are used to limit the number of unacknowledged packets
- Two base: send_base, rcv_base



Transport Layer

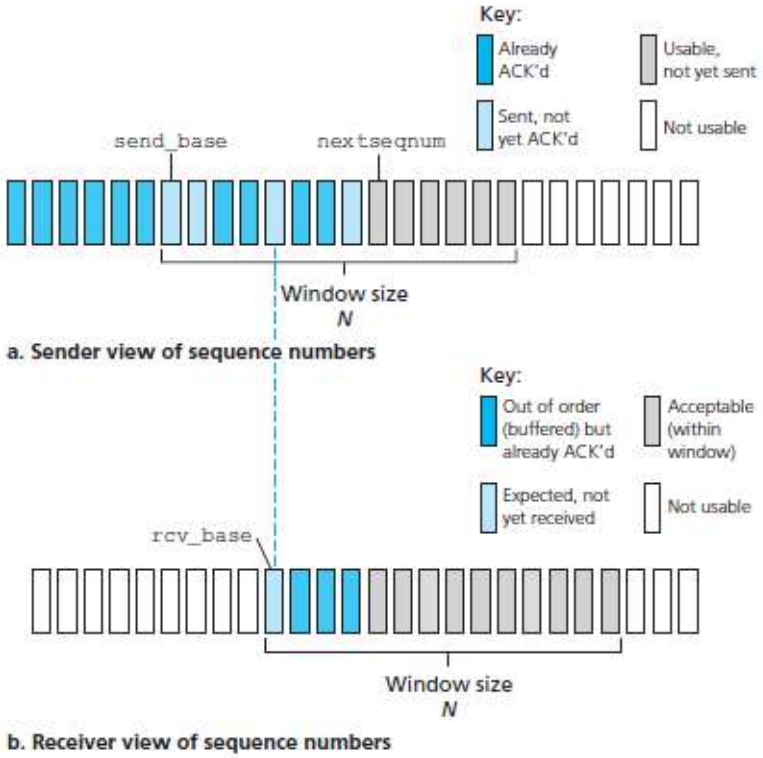
Reliable Data Transfer: Selective Repeat

Selective-repeat (SR) protocols:

- the sender retransmit only those packets that were likely received in error (lost or corrupted)
- Unlike GBN, out-of-order packets are buffered and acknowledged by the receiver instead of discarded.

Approach:

- Two windows of size N are used to limit the number of unacknowledged packets
  - send_base: sequence number at the lower edge of the sender window (oldest not-yet-acknowledged position).
  - rcv_base: sequence number at the lower edge of the receiver window (next packet needed for in-order delivery).



Transport Layer

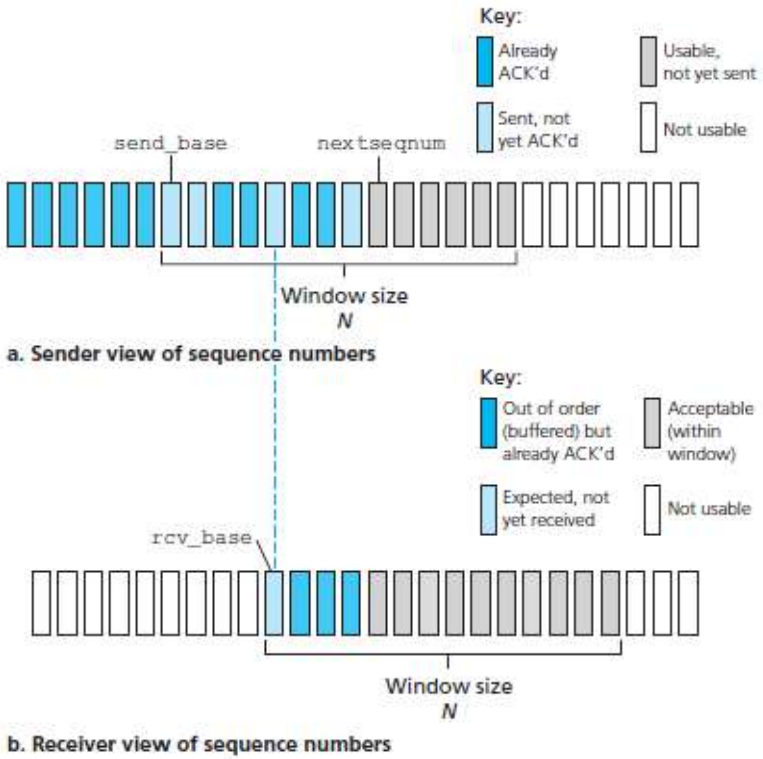
Reliable Data Transfer: Selective Repeat

Sender:

- Sending data: if the seq. number falls inside the window, packetise and send it
- One timer per packet: On timeout, resend that packet
- ACK n: mark n received; if n was the base of the send windows, slide it to the next unacknowledged number

Receiver:

- A packet inside the window: acknowledge it, and buffer it if it is new.
- If it is the base of the rcv window: deliver it and every buffered packet consecutive with it.
- A packet just below the window: acknowledge it again, and drop it.



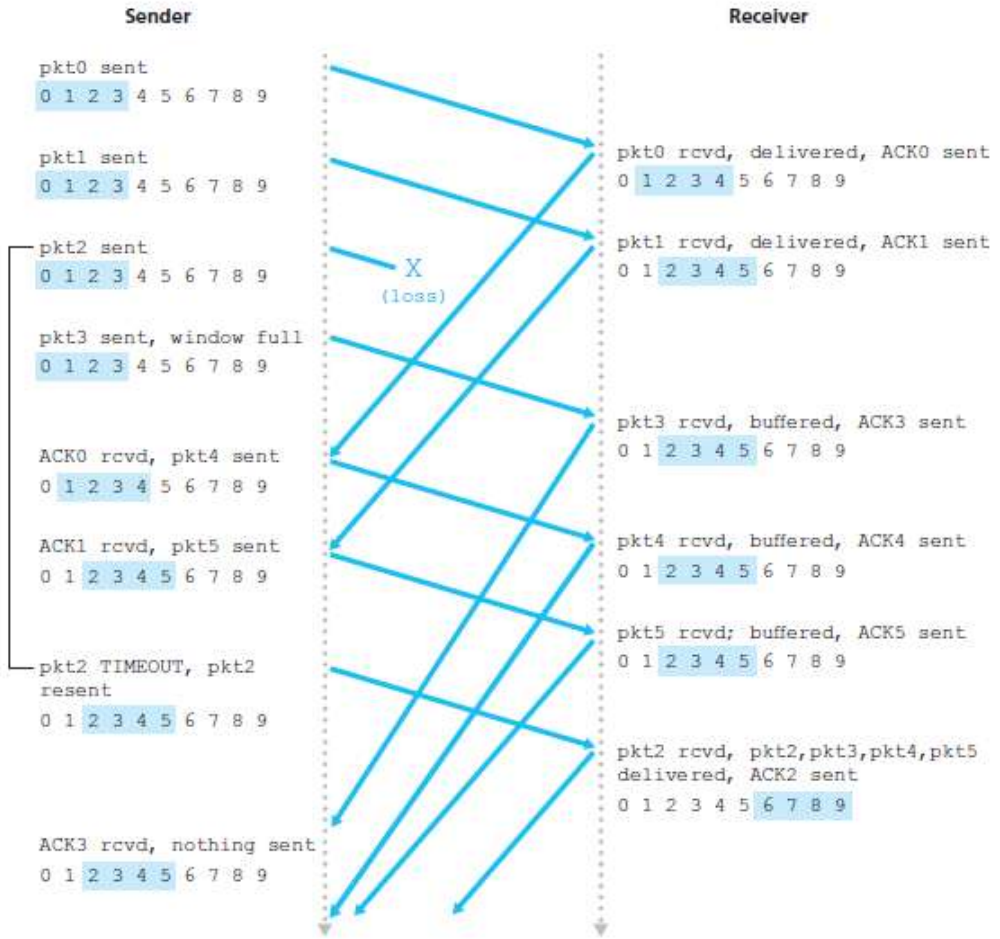
Transport Layer

Reliable Data Transfer: Selective Repeat

Example of SR operation in the presence of lost packets

- The receiver initially buffers pkt3, pkt4, and pkt5, while waiting for pkt2 (lost) to be retransmitted.
- Only pkt2 is retransmitted, on its own timer. Nothing else goes out twice.
- When it arrives, four packets go up to the application in one move, in order.

- Here we avoid to resend pkt3, pkt4, pkt5, in so avoiding additional transmission loss or errors.



Transport Layer

Reliable Data Transfer: Selective Repeat

Acknowledge what is already below your window

- The receiver has that packet, delivered long ago. Why answer at all?

Transport Layer

Reliable Data Transfer: Selective Repeat

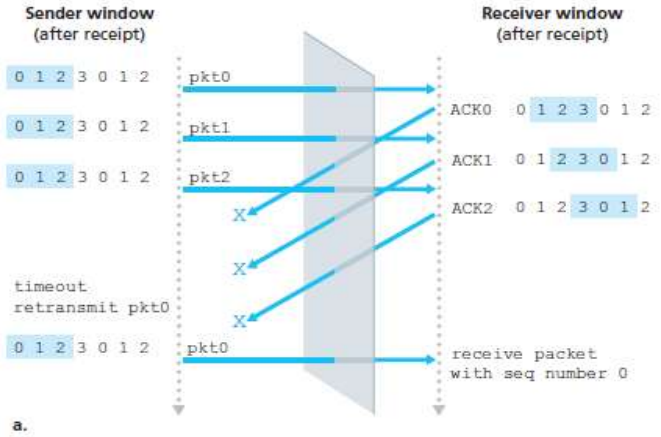
Acknowledge what is already below your window

- The receiver has that packet, delivered long ago. Why answer at all?
- Because the sender does not know. If the acknowledgement for the oldest unacknowledged packet was lost, the sender will keep retransmitting it, and its window will never move forward.
- The two sides do not share a view of what has arrived, and in selective repeat that gap is permanent, not temporary.
- Silence is never an acceptable answer to a retransmission: it is indistinguishable from a second loss.
- So the receiver re-acknowledges and drops.

Transport Layer

Reliable Data Transfer: Selective Repeat – window size

- One issue in SR: the window size is related to the sequence number, **and sequence number is finite**.
- In this example we have four packets, **a max sequence number of 3** and a window size of 3.
- Let's assume packets 0 to 2 are correctly received and their ACK have been sent (but not yet received).
  - The **receiver's window moves to 6th packet** (i.e., to [3, 0, 1]) even if ACK are not received for certain.
  - **Case a**: the ACKs for the first three packets are lost and the sender retransmits these packets.
    Is packet 0 new or old?



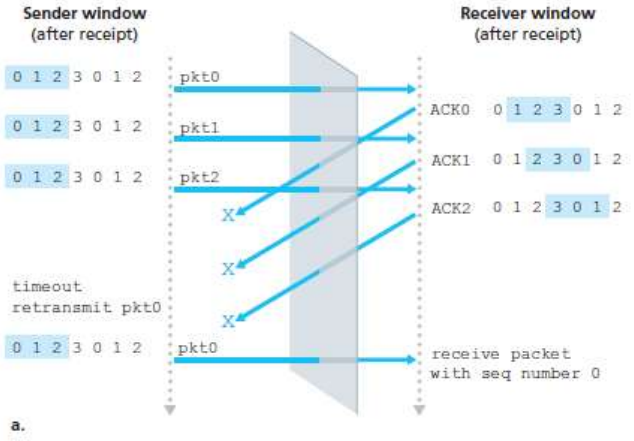
Transport Layer

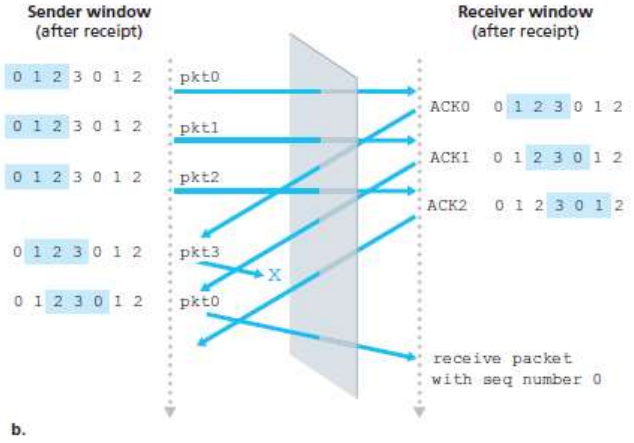
Reliable Data Transfer: Selective Repeat – window size

- One issue in SR: the window size is related to the sequence number, **and sequence number is finite**.

- In this example we have four packets, **a max sequence number of 3** and a window size of 3.

- Let's assume packets 0 to 2 are correctly received and their ACK have been sent (but not yet received).
  - The **receiver's window moves to 6th packet** (i.e., to [3, 0, 1]) even if ACK are not received for certain.
  - **Case b**: the ACKs are received, packets 3 and 0 are sent but packet 3 is lost. *Is packet 0 new or old?*



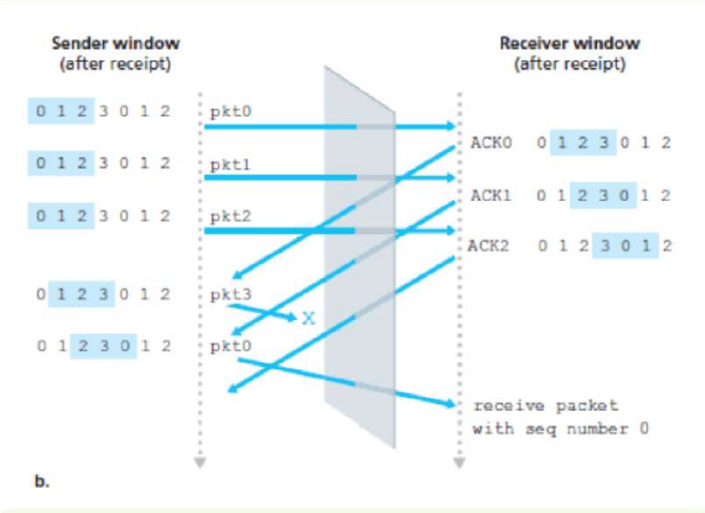


Transport Layer

Reliable Data Transfer: Selective Repeat – window size

Case b: the acknowledgements all arrive.

- The sender sends 3, 0, 1 — and pkt3 is lost while pkt0 gets through.
- The receiver sees a packet numbered 0, inside its window, and this time it is new data.
- It cannot see the sender: both cases produce the same sequence of arrivals, so the two are indistinguishable: *Is packet 0 new or old?*



Transport Layer

Selective Repeat: Maximum Window Misalignment

- The sender can transmit at most **N unacknowledged packets**.
- If all N packets are received but their ACKs are lost, the receiver advances by N positions while the sender remains blocked.
- **The receiver cannot advance further**, since the sender cannot transmit new packets without receiving ACKs.

- Therefore, the maximum window misalignment is **N positions.**

Question: How can the maximum window misalignment constraint help us solve the ambiguity problems discussed earlier?

Transport Layer

Selective Repeat: ambiguity problem

Question: How can the maximum window misalignment constraint help us solve the ambiguity problems discussed earlier?

- **Maximum window misalignment:** N positions (all N packets received, but ACKs lost).
- The **sequence number space** M must prevent **overlap between old and new windows**.
- Therefore: $M \geq 2N$

